

# Structure with XML tags

---

Video Link: [Structure with XML tags](#)

When you're building prompts that include a lot of content, Claude can sometimes struggle to understand which pieces of text belong together or what different sections are supposed to represent. XML tags provide a simple way to add structure and clarity to your prompts, especially when you're interpolating large amounts of data.

## Why Structure Matters

Consider a prompt where you need to analyze 20 pages of sales records. Without clear boundaries, Claude might have trouble distinguishing between your instructions and the actual data you want analyzed.

### Providing Structure

**Use XML tags to separate distinct portions of the prompt**

- Most useful when including a lot of context
- Help serve as delimiters for Claude

ANTHROPIC

```
Write a one page decision report to troubleshoot why a sales team's numbers have dropped 30% last quarter.
```

```
Here are the last 20 pages of our sales records: {sales_records}
```

```
Follow these steps:
```

1. Compare current vs previous market metrics
2. Identify relevant industry changes
3. Analyze individual team member performance
4. Consider recent organizational changes
5. Review customer feedback

The example above shows how unclear boundaries can make it difficult for Claude to parse your intent. By wrapping the sales records in XML tags like `<sales_records>` and `</sales_records>`, you create clear delimiters that help Claude understand the structure of your prompt.

## Providing Structure

Use XML tags to separate distinct portions of the prompt

- Most useful when including a lot of context
- Help serve as delimiters for Claude

ANTHROPIC

```
Write a one page decision report to troubleshoot why a sales team's numbers have dropped 30% last quarter.
```

```
Here are the last 20 pages of our sales records:
```

```
<sales_records>
{sales_records}
</sales_records>
```

```
Follow these steps:
```

1. Compare current vs previous market metrics
2. Identify relevant industry changes
3. Analyze individual team member performance
4. Consider recent organizational changes
5. Review customer feedback

## Practical Example: Code and Documentation

Here's a more dramatic example of why XML tags matter. If you ask Claude to debug code using provided documentation, mixing everything together creates confusion:

## Providing Structure

Not Great

```
Debug my code below using the provided documentation.

from datavortex import Pipeline, DataSource

def process_data(input_file, output_file):
    pipeline = Pipeline()
    source = DataSource.from_csv(input_file)

# Creating a data source from data vortex
csv_source = DataSource.from_csv("data.csv")
```

ANTHROPIC

Better!

```
Debug my code below using the provided documentation.

<my_code>
from datavortex import Pipeline, DataSource

def process_data(input_file, output_file):
    pipeline = Pipeline()
    source = DataSource.from_csv(input_file)
</my_code>

<docs>
# Creating a data source from data vortex
csv_source = DataSource.from_csv("data.csv")
</docs>
```

The "Not Great" version makes it nearly impossible to tell what's code versus documentation. The "Better" version uses `<my_code>` and `<tags>` to create clear boundaries.

## Custom Tag Names

You don't need to use official XML tags. Create descriptive names that make sense for your content:

- `<sales_records>` is better than
- `<athlete_information>` clearly identifies user details
- `<my_code>` and separate different types of content

The more specific and descriptive your tag names, the better Claude can understand the purpose of each section.

## When to Use XML Tags

XML tags are most useful when:

- Including large amounts of context or data
- Mixing different types of content (code, documentation, data)
- You want to be extra clear about content boundaries
- Working with complex prompts that interpolate multiple variables

Even for shorter content, XML tags can help serve as delimiters that make your prompt structure more obvious to Claude.

## Real-World Application

In practice, you might structure a prompt like this:

```
<athlete_information>
- Height: 6'2"
- Weight: 180 lbs
- Goal: Build muscle
- Dietary restrictions: Vegetarian
</athlete_information>
```

Generate a meal plan based on the athlete information above.

This makes it crystal clear that the height, weight, goal, and restrictions are all related athlete data that should be considered together when generating the meal plan.

While you might not see dramatic improvements with simple prompts, XML tags become increasingly valuable as your prompts grow more complex and include larger amounts of varied content.